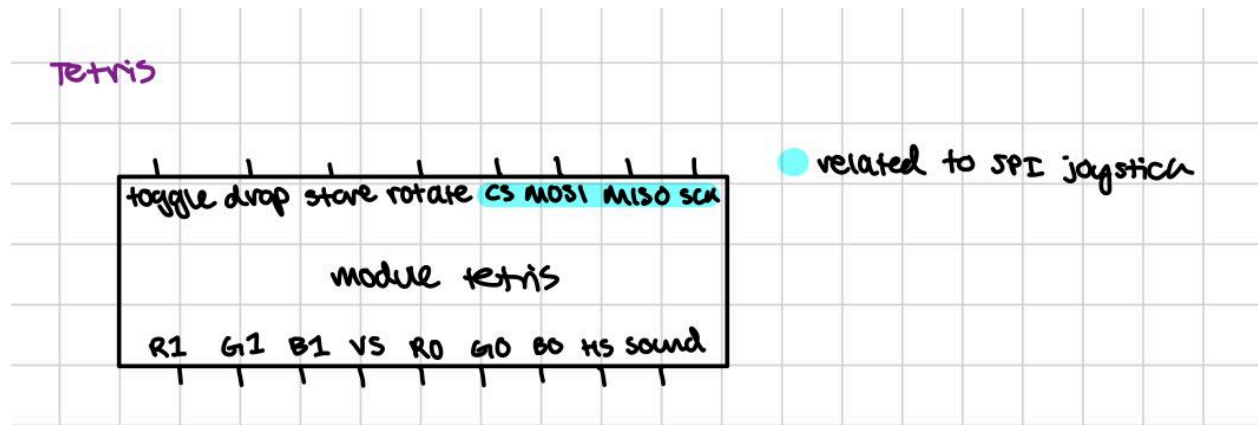


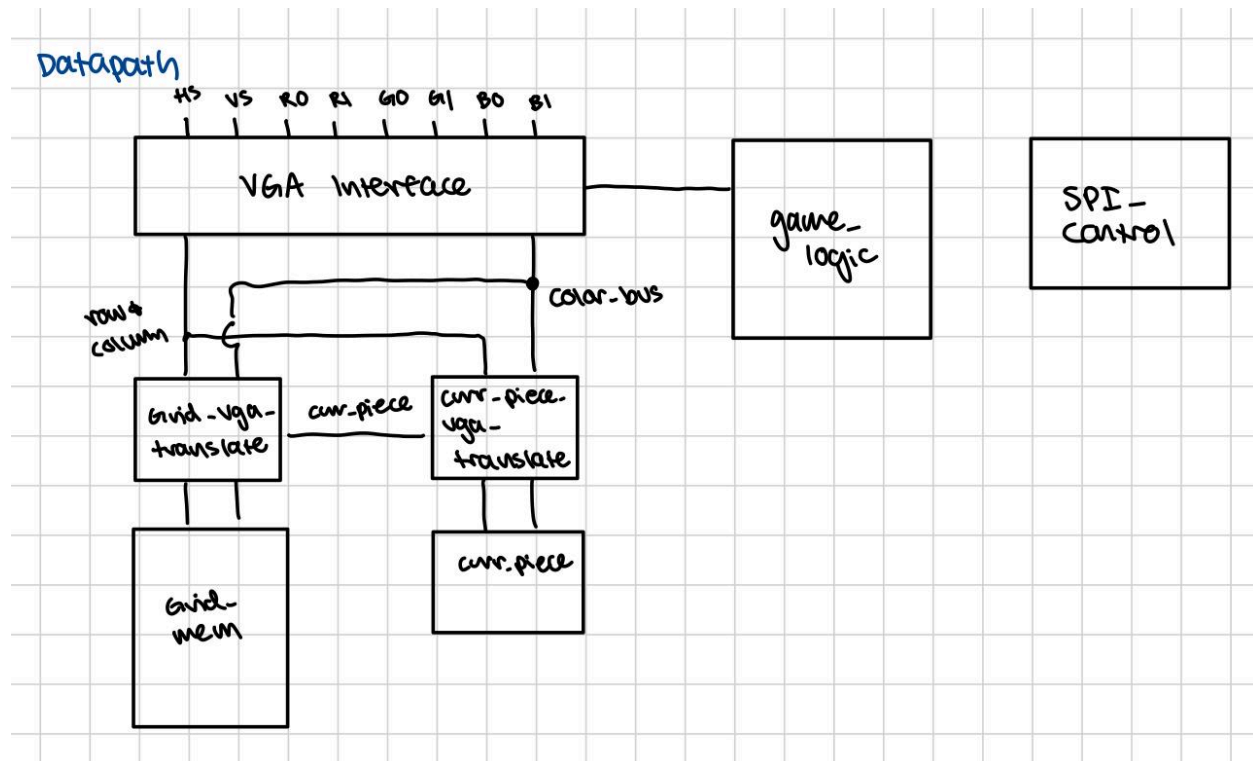
Overall Module



The main strategy of this design is to use the fact that the VGA cycle is really long to allow for game state to be updated over multiple clock cycles instead of one.

Disclaimer: this is kind of bad I'm sorry

Datapath Overview



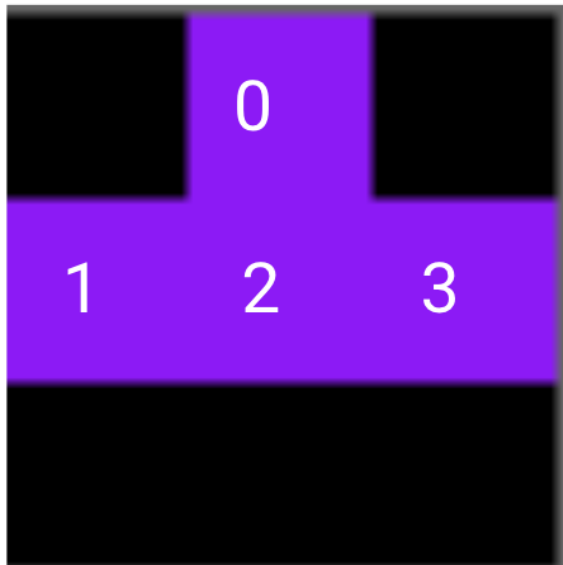
Module descriptions:

- VGA interface: a modified version of the 240 vga interface with smaller red, green, and blue signals. The interface keeps track of VGA timing.
- grid_vga_translate: based on the row and column outputted by the vga interface, drives the color_bus with the pixel at that location. Does not drive the bus if the curr_piece_vga_translate is active.
- curr_piece_vga_translate: based on the row and column outputted by the vga interface, drives the bus with the curr_piece color if the row and columns match.
- grid_mem: registers that keep track of the committed pixels
- curr_piece: keeps track of the current piece statistics (e.g. location, type, orientation).
- game_logic: control logic that drives updating of curr_piece and grid_mem.
- SPI_control: interfaces with the spi joystick. I've been looking at the documentation for the spi controller but its really bad (https://digilent.com/reference/_media/reference/pmod/digilent-pmod-interface-specification.pdf)

[illegible]

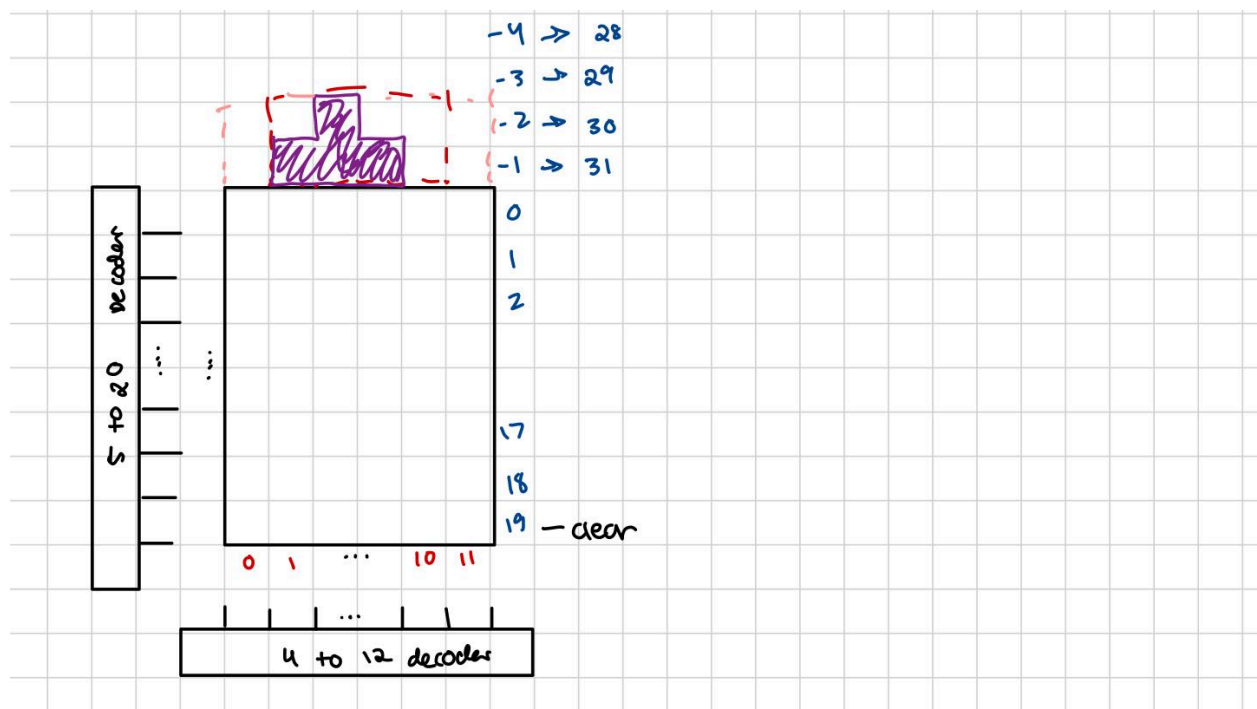
The image displays a 6x4 grid of 24 small images. Each image features a white dot on a black background, with a colored shape (cyan, blue, orange, yellow, green, purple, or red) positioned around the dot. The shapes are arranged in a pattern that suggests a sequence or a logic puzzle. The shapes are arranged in a grid that is 6 rows high and 4 columns wide. The shapes are arranged in a grid that is 6 rows high and 4 columns wide. The shapes are arranged in a grid that is 6 rows high and 4 columns wide.

To extract each of the pixels, I have a pixel translator module that takes in all of the metadata plus a pixel number:



The pixel numbers are ordered based on which need to be displayed first on the vga. Next_state logic (orientation, x, y) is determined by a priority encoder that samples user inputs (joystick, drop) whenever the game progresses.

Grid Mem Overview



This is how the grid is numbered.

```

graph TD
    IDLE((IDLE)) -- "rst" --> IDLE
    IDLE -- "play/new-piece" --> move((move))
    move -- "pixel_num < 3 / pixel_en" --> move
    move -- "pixel_num < 3 / pixel_en" --> load((load))
    load -- "pixel_num < 3 / pixel_en we" --> load
    load -- "pixel_num == 3 / pixel_en we" --> wait((wait))
    wait -- "rst" --> wait
    wait -- "play" --> clr((clr))
    clr -- "clear/store new-piece" --> clr
    clr -- "~clear & ~store" --> move
    
```

State variable structure:

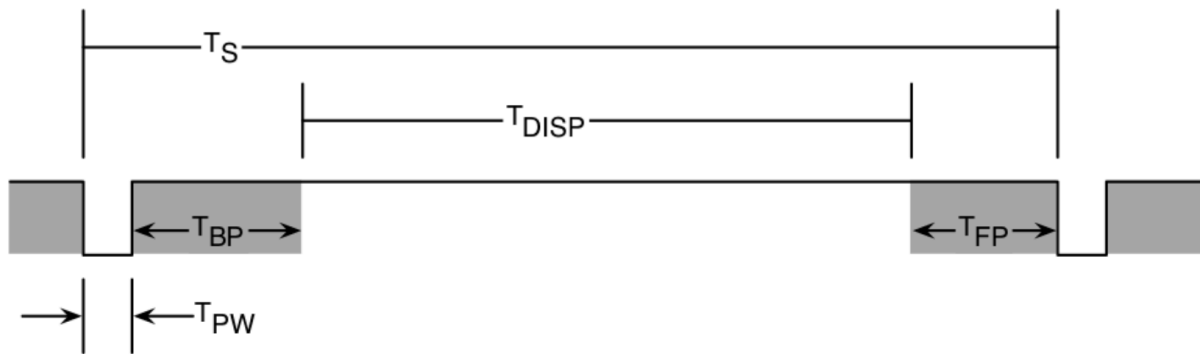
pixel_en	pixel_num
rst	cnt

Current state: pixel_num

- IDLE: just state that reset goes to.
- MOVE: extract each of the pixels of the current piece's next state and determine if they are valid (within bounds, overlapping with any committed pieces).
- LOAD: if any of the pixels of the current piece's next state are not valid because of a pixel below, that means it should be committed and so we load it into our memory.
- WAIT: wait for the next game cycle.
- CLR: handles clearing of filled rows. If the current move is a store, load a different piece and go back to WAIT state.

- IDLE: just state that reset goes to.
- MOVE: extract each of the pixels of the current piece's next state and determine if they are valid (within bounds, overlapping with any committed pieces).
- LOAD: if any of the pixels of the current piece's next state are not valid because of a pixel below, that means it should be committed and so we load it into our memory.
- WAIT: wait for the next game cycle.
- CLR: handles clearing of filled rows. If the current move is a store, load a different piece and go back to WAIT state.

VGA timing courtesy of Professor Nace.



The general timing is at T_{FP} if the game state needs to be updated, sample the SPI joystick + button input and then assert play. This approach will hopefully give as much time as possible for game logic to finish before the next T_{DISP} .

Testing

At first, I thought about making an interactive testbench since Python has an `input()` function. But, since all the coroutines are asynchronous I don't know how well that would've gone (and I probably would also have to download Verilator onto my computer). The second thing I thought of was to write a golden model in Python (which I have a semi-one from 15-112), put in some inputs and capture the grid after each input and compare it to the simulation's output.